TaskForge

A Kubernetes-Native Task List Web Application

Project Proposal — DevOps Engineering Module

Project Description

TaskForge is a deliberately small full-stack application: a single-entity task list where a user can create, view, update, and delete tasks — no authentication, no multi-user collaboration, no secondary domain objects. The application layer is kept to the minimum needed to prove real frontend-backend-database connectivity; the project's actual weight sits on the **end-to-end DevOps lifecycle** around that small app: infrastructure provisioned as code, configuration automated, and the app deployed and orchestrated on Kubernetes with a self-hosted database — with no manual, hand-run server setup at any stage.

Objectives

- Provision reproducible cloud infrastructure declaratively using **Terraform**.
- Automate OS-level configuration and dependency installation (Docker, container runtime, Kubernetes prerequisites) using **Ansible** — no manual server steps.
- Containerize the frontend, backend, and database, and orchestrate them on a **Kubernetes** cluster with health checks, rolling updates, and autoscaling.
- Run **MongoDB self-hosted** inside the cluster (official Docker image, not a managed cloud DB service), backed by persistent storage.
- Expose the application securely through an Ingress controller with a single entry point.

Technical Specifications

Layer	Technology	Role in the System
Frontend	React	Single-page UI to list, add, edit, complete and delete tasks; served as static assets via Nginx.
Backend	Express.js	REST/JSON API (Node.js) exposing CRUD endpoints for the task resource.
Database	MongoDB	Self-hosted via the official `mongo` container image; stores task documents.
Containerization	Docker	Frontend, backend, and database each packaged as a Docker image.
Orchestration	Kubernetes	Deployments, Services, Ingress, PVCs, ConfigMaps/Secrets, HPA.
Infrastructure as Code	Terraform	Provisions VMs/network/cluster nodes on the target cloud/on-prem provider.
Config. Management	Ansible	Installs Docker/container runtime, joins nodes to the cluster, applies base hardening.

Key Non-Functional Requirements

- **Reproducibility:** entire environment stood up from Terraform + Ansible + Kubernetes manifests, no undocumented manual steps.
- **Statefulness handled explicitly:** MongoDB runs as a `StatefulSet` with a PVC (`PersistentVolumeClaim`) so data survives pod restarts/rescheduling.
- **Configuration & secrets:** environment-specific values via Kubernetes `ConfigMaps`/`Secrets`, not baked into images.
- **Resilience:** liveness/readiness probes on frontend and backend Deployments; rolling updates with zero downtime.
- **Scalability:** backend Deployment scales horizontally behind its Service/Ingress independent of the frontend and database.

System Architecture

Provisioning → Configuration → Orchestrated Runtime

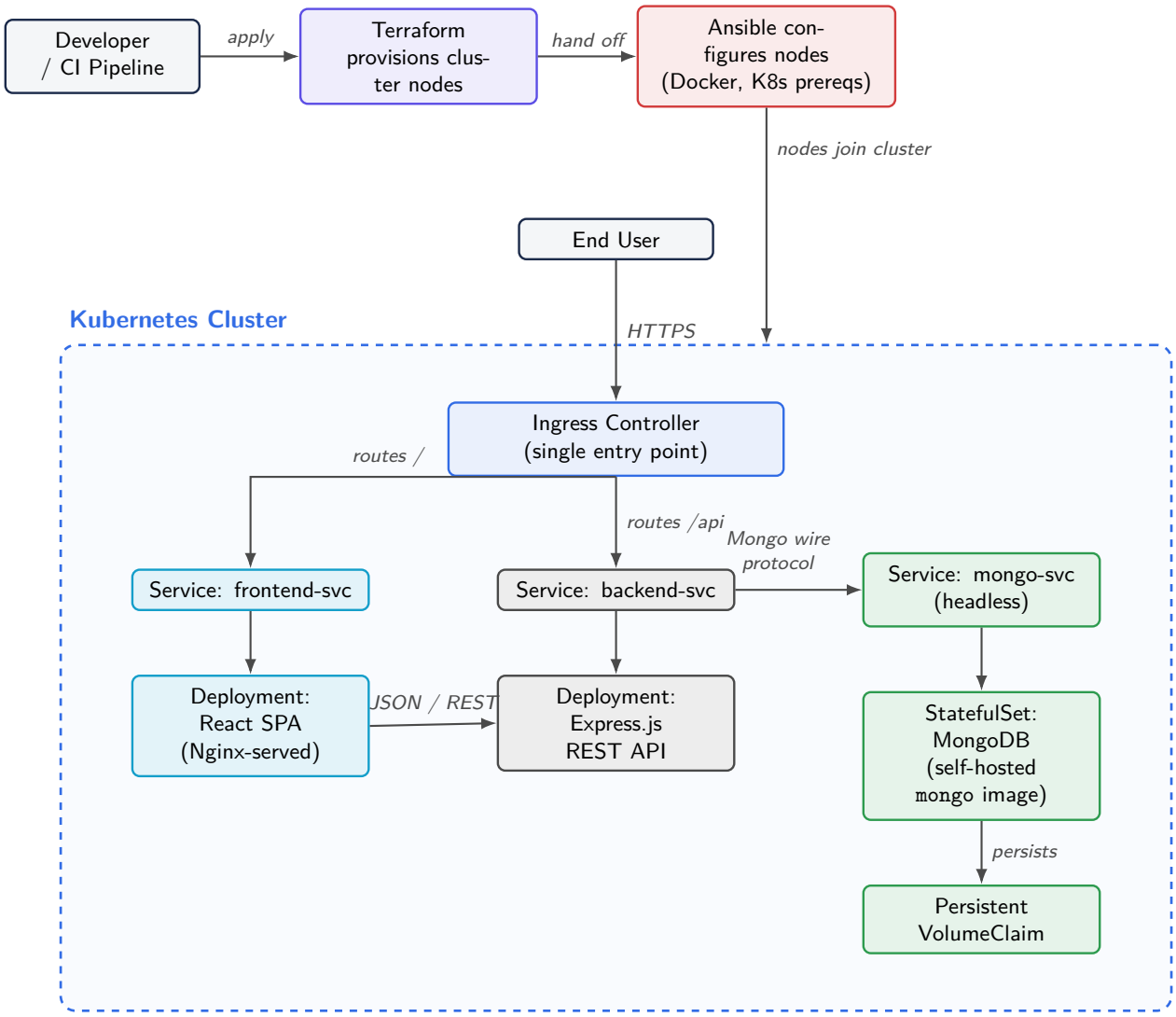

Diagram Notes

- **Provisioning & config (top):** Terraform creates the cluster nodes / networking; Ansible then configures each node (container runtime, Kubernetes prerequisites) and joins it to the cluster.
- **Runtime (dashed box):** everything inside the Kubernetes cluster boundary is deployed via manifests — Ingress, three Deployments/StatefulSets, their Services, and one PVC for MongoDB's data directory.
- **Traffic path:** the user only ever talks to the Ingress; it fans requests out to the frontend Service (static routes) and backend Service (`/api` routes), which in turn talks to MongoDB over the internal cluster network — nothing but Ingress is exposed externally.